Machine Learning in Finance
Summer 2025

Machine Learning in Finance - Project

GROUP MLTAMBE
DECAUX Théodore 329169
VELAY Mahé 345882
BERETZ Benjamin 345638
BOURGON Elias 346661
ROBERT Adélaïde 346471

Professor: Malamud Semyon

June 15, 2025

Contents

1 Introduction and problem statement **1**

2 Description of Data Preprocessing **1**
2.a General Preprocessing . . . 1
2.b Dataset-Specific Adjustments . . . 1
2.c Merging Datasets . . . 1
2.d Lagging Predictors . . . 1

3 Feature Engineering and Dataset Preparation **2**
3.a Construction of Time-Series Features and Temporal Splitting . . . 2
3.b Temporal Splitting with Firm-Level Filtering . . . 2

4 Explanation of the Predictive Models Used **2**
4.a Overview and Feature Selection Strategy . . . 2
4.a.1 Motivation for Feature Filtering . . . 2
4.b Tree-Based Model: XGBoost . . . 3
4.b.1 Model Description . . . 3
4.b.2 Why XGBoost for Financial Data . . . 3
4.b.3 Training Procedure . . . 3
4.b.4 Feature Selection . . . 3
4.c Linear Models . . . 3
4.c.1 Linear Regression . . . 3
4.c.2 Implementation Strategy . . . 3
4.c.3 Limitations . . . 3
4.d Ridge Regression with Cross-Validation . . . 4
4.d.1 Model Formulation . . . 4
4.d.2 Implementation and Cross-Validation . . . 4
4.d.3 Interpretation . . . 4
4.e Lasso Regression with Cross-Validation . . . 4
4.e.1 Model Formulation . . . 4
4.e.2 Implementation and Cross-Validation . . . 4
4.e.3 Strengths and Usefulness . . . 4
4.f Multi Layer Perception (MLP) . . . 4
4.f.1 Model . . . 4
4.f.2 Loss function . . . 5
4.f.3 Training the model . . . 5

5 Evaluation methodology and performance metrics **5**
5.a Linear models . . . 5
5.a.1 OLS . . . 5
5.a.2 Ridge CV . . . 6
5.a.3 Lasso . . . 6
5.b XGBoost . . . 6
5.c Multi Layer Perception (MLP) . . . 6
5.c.1 Evaluation strategy . . . 6
5.c.2 Parameters and results . . . 7

6 Conclusion and Possible Extensions **7**

7 Appendix **8**
7.a Train-Test Split distribution . . . 8
7.b OLS graphs . . . 9
7.c Ridge graphs . . . 10
7.d Lasso graphs . . . 11
7.e MLP graphs . . . 12

1 Introduction and problem statement

In the context of modern financial markets, the ability to forecast asset returns using firm-level data has become a key priority for quantitative investment strategies. Machine learning offers flexible tools for modeling nonlinear relationships and high-dimensional interactions, which are often missed by traditional linear models such as CAPM or Fama-French.

This project aims to forecast monthly U.S. stock returns using financial characteristics available at the time of prediction. The target variable is taken from the Monthly CRSP dataset, while predictors are drawn from the Quarterly Compustat fundamentals and JKP factor characteristics (from homework 2), both offering rich information on firm performance and risk exposures. To ensure consistency and prevent forward-looking bias, these sources are merged and aligned using standardized linking tables and time-based filters. Additionally, only firms with sufficient historical data coverage are retained across the training, validation, and test sets, ensuring a stable panel structure suitable for return prediction.

Once the data is cleaned and properly structured, the focus turns to building and evaluating predictive models. Several approaches are tested, including a deep learning model. To better reflect financial objectives, one of the models incorporates a custom loss function with two complementary objectives (MSE and Sharpe Ratio). To avoid data mining and ensure robustness, the dataset is split chronologically into training, validation, and test subsets. Model performance is evaluated using both statistical metrics and financial relevance, with the goal of assessing the practical value of machine learning in realistic return forecasting scenarios.

2 Description of Data Preprocessing

2.a General Preprocessing

The JKP factor dataset and the Compustat firm fundamentals undergo consistent preprocessing: columns with over 30% missing values are dropped, dates are standardized, duplicate firm-date entries removed, data is sorted by firm and time, and missing values are imputed using forward and backward filling within each firm. Numerical features are winsorized at the 1% and 99% quantiles to limit the influence of outliers.

2.b Dataset-Specific Adjustments

In addition to shared steps, each dataset requires specific handling. The JKP dataset includes both `gvkey` and `permno`, but some mappings are incomplete; a linking table from WRDS is used to recover missing pairs. This is essential, as Compustat relies on `gvkey`, while CRSP uses `permno`. The JKP data, reported monthly with stock identifiers, can be merged directly with CRSP. Compustat, reported quarterly using `gvkey`, requires special attention to ensure temporal consistency.

2.c Merging Datasets

After preprocessing, JKP, Compustat, and CRSP return data are merged. The merge is restricted to firms present in both JKP and Compustat, then aligned using a time-aware join (`pd.merge_asof`) that matches each monthly JKP observation to the latest available Compustat record. The result is then joined with CRSP using `permno` and `date`. The merged data is sorted chronologically, rows with missing values removed, and observations prior to 1985 excluded to ensure sufficient coverage.

2.d Lagging Predictors

To preserve causality, all numeric predictors are lagged by one period within each firm, so that only information available at time t is used to predict returns at $t+1$. Rows with missing lagged features are dropped, yielding a clean, temporally ordered panel. The input matrix X includes lagged financial and factor features; the target y is the next-month stock return and firm identifier.

3 Feature Engineering and Dataset Preparation

3.a Construction of Time-Series Features and Temporal Splitting

To enhance the predictive power of our models for future stock returns, we developed a function that automatically generates explanatory variables from historical firm-level time-series. All features are strictly backward-looking to ensure temporal causality and prevent information leakage.

For return-related variables, we compute moving averages over different horizons (1, 3, 6 months) to capture momentum effects, as well as "streak" indicators counting consecutive positive or negative returns. Volatility indicators are constructed using rolling windows to reflect historical risk. This logic is extended to fundamental variables such as market equity, asset turnover, and beta, for which we compute smoothed values and ratios comparing current levels to recent averages—mirroring valuation signals used in multifactor models.

Features are cleaned using forward and backward fill, and variables with persistent missing values are dropped. Out of 192 initial features, only 42 passed density and consistency thresholds. Most discarded features involved long lookback horizons or unstable constructs such as RSI, drawdowns, skewness, or kurtosis.

Despite theoretical appeal, many engineered features added little incremental value. Transformations on ratios like Return on Equity or Net Operating Assets often introduced noise due to semantic mismatches or redundancy with existing JKP variables. This was especially problematic in high-capacity models like neural networks, where dimensionality increases the risk of overfitting. Since the JKP dataset already includes a rich set of economically motivated indicators, we ultimately excluded the engineered features to retain a leaner, more interpretable, and empirically robust feature set.

3.b Temporal Splitting with Firm-Level Filtering

To ensure robust evaluation and mitigate survivorship bias, we applied a time-series split: the dataset is divided chronologically into training (70%), validation (10%), and test (20%) sets. All firms (`permno`) observed during the training window are retained, regardless of whether they appear later.

After this chronological split, we enforce a firm consistency constraint on validation and test sets: only firms already present during training are included in these subsets. This prevents the model from being evaluated on previously unseen entities, which would be unrealistic in a predictive context. Importantly, the training set includes all firms observed before the cutoff, maximizing the learning signal.

This two-step procedure addresses common issues in financial datasets, where firms may enter or exit due to IPOs, delistings, or incomplete records. Without filtering, models could be evaluated on firms lacking any historical context. Our approach ensures consistent and realistic evaluation while making full use of the training data's breadth. Figure 1

4 Explanation of the Predictive Models Used

4.a Overview and Feature Selection Strategy

All the models used in this project are either covered in the course or represent natural extensions of the techniques studied (XGBoost as a boosted-tree extension of decision trees, and MLP as a standard deep learning model).

4.a.1 Motivation for Feature Filtering

To ensure consistency and reduce the risk of overfitting, we first use XGBoost to rank all features by importance and retain only the top 70. This feature selection step provides a compact and informative subset of predictors, which is then reused across all subsequent models. By filtering the data in this way, we improve training stability and comparability between models while preserving the most predictive signals.

4.b Tree-Based Model: XGBoost

4.b.1 Model Description

XGBoost (Extreme Gradient Boosting) is a powerful ensemble method based on additive decision trees. Each successive tree is trained to correct the residual errors of the previous ones. This iterative approach enables the model to capture complex, nonlinear relationships between inputs and outputs.

4.b.2 Why XGBoost for Financial Data

We chose XGBoost due to its strong performance on tabular data, its robustness to missing values and feature scaling, and its ability to handle high-dimensional predictors. These properties make it particularly suitable for financial datasets, where relationships between firm-level characteristics and returns are often nonlinear and noisy [**?**].

4.b.3 Training Procedure

In our implementation, XGBoost is trained on chronologically ordered training data using the `xgb.train` API. Early stopping is applied using a temporally consistent validation set to prevent overfitting and ensure time-aware tuning. Data is first converted into XGBoost's optimized `DMatrix` structure to improve efficiency.

4.b.4 Feature Selection

After training, we extract feature importances using the `weight` metric, which reflects how often each feature is used in tree splits. The top 70 features are selected and reused by all subsequent models. This step reduces dimensionality, simplifies downstream training, and helps with interpretation.

Although alternative importance metrics such as `gain` (which measures the average reduction in loss attributable to each feature) are theoretically appealing, we observed that selecting features based on `weight` led to more robust out-of-sample performance, particularly for the MLP. In our experiments, using `gain` resulted in a lower Sharpe ratio, suggesting that it may have promoted features that were overly specific or regime-dependent. In contrast, `weight`-based selection favored features that were used more frequently across trees, indicating broader applicability and greater stability. This was especially beneficial for neural networks, which are more sensitive to noise and overfitting. As a result, we adopted `weight` as our default importance metric for feature selection.

4.c Linear Models

4.c.1 Linear Regression

Linear regression serves as a baseline model to estimate future stock returns. It assumes a linear relationship between predictors and the target return:

$$R_{i,t+1} = \beta_0 + \beta_1 X_{i,t}^{(1)} + \cdots + \beta_p X_{i,t}^{(p)} + \varepsilon_{i,t+1}$$

The model is trained by minimizing the mean squared error (MSE) between predicted and actual returns. However, it does not capture interactions or nonlinear effects.

4.c.2 Implementation Strategy

To support both prediction and interpretation, we use two implementations: `scikit-learn`'s `LinearRegression` for model training and prediction, and `statsmodels.OLS` for generating statistical outputs such as p-values and confidence intervals.

4.c.3 Limitations

Linear regression is simple and interpretable, but its strong assumptions (linearity, additivity, homoscedasticity) may limit performance in complex financial settings.

4.d Ridge Regression with Cross-Validation

4.d.1 Model Formulation

Ridge regression augments linear regression with an ℓ_2 penalty to control overfitting:

$$\min_{\boldsymbol{\beta}} \sum_{i,t} (R_{i,t+1} - \beta_0 - \boldsymbol{X}_{i,t}^\top \boldsymbol{\beta})^2 + \lambda \|\boldsymbol{\beta}\|_2^2$$

This helps prevent overfitting, especially when predictors are highly correlated.

4.d.2 Implementation and Cross-Validation

We use `scikit-learn`'s `RidgeCV` to automatically select the optimal regularization parameter λ. A time-aware `TimeSeriesSplit` strategy with five expanding folds ensures chronological consistency.

4.d.3 Interpretation

Ridge regression improves generalization while preserving all predictors, making it useful when multicollinearity is present but full variable inclusion is desired.

4.e Lasso Regression with Cross-Validation

4.e.1 Model Formulation

Lasso regression promotes sparsity by penalizing the ℓ_1 norm of the coefficients:

$$\min_{\boldsymbol{\beta}} \sum_{i,t} (R_{i,t+1} - \beta_0 - \boldsymbol{X}_{i,t}^\top \boldsymbol{\beta})^2 + \lambda \|\boldsymbol{\beta}\|_1$$

This allows Lasso to perform both regularization and variable selection simultaneously.

4.e.2 Implementation and Cross-Validation

We use `scikit-learn`'s `LassoCV` with a time-aware `TimeSeriesSplit` and a predefined grid of λ values. Each fold uses only past data to predict future values, mimicking a realistic forecasting scenario.

4.e.3 Strengths and Usefulness

Lasso is particularly useful for high-dimensional financial data, where many variables may be redundant. It promotes model simplicity by eliminating weak predictors and retaining only those with strong signals.

4.f Multi Layer Perception (MLP)

Multi-Layer Perceptrons (MLPs) offer a compelling approach to stock return prediction due to their ability to capture complex, non-linear relationships that traditional linear models often miss. Financial markets are inherently non-linear systems where multiple factors interact in intricate ways—from macroeconomic indicators and company fundamentals to market sentiment and technical patterns.

4.f.1 Model

The first step was creating the class for our MLP's model. We designed a simple MLP providing us flexibility in the framework for neural network construction.

Our Network was build combining the number of features, our hidden layers dimension and an output of dimension 1. The model then iterates through these dimensions, constructing fully connected linear layers between each pair of consecutive sizes.

Between the hidden layers, we incorporate **ReLU activation** functions to introduce non-linearity, enabling the network to learn complex patterns in the data. Additionally, we apply **dropout regularization** after each hidden layer to prevent overfitting.

The **weight initialization** strategy employs a normal distribution with carefully chosen parameters. Weights are initialized with a mean of 0 and standard deviation of 0.1, while biases use a smaller standard deviation of 0.01. This initialization scheme helps ensure stable training from the start and prevents gradient-related issues during optimization.

Finally, the forward pass simply feeds the input through the sequential network and removes the extra dimension from the output, returning a single prediction value for each sample.

4.f.2 Loss function

The model is trained with a hybrid objective

$$\mathcal{L} \;=\; w_{\mathrm{MSE}}\,\mathrm{MSE} \;-\; w_{\mathrm{S}}\,\mathrm{Sharpe},$$

which balances statistical accuracy and trading quality:

- **MSE** enforces correct magnitude and direction of next-period returns;
- **Sharpe term** rewards forecasts that improve risk-adjusted performance.

Appropriate weighting lets the network stay predictive while favouring tradeable signals.

Reflexion steps
Before arriving at this hybrid approach, we experimented with each loss component in isolation.
When using only MSE, the model achieved reasonable prediction accuracy but lacked any consideration for risk-adjusted performance—it might predict returns accurately but in ways that would lead to poor trading strategies with high volatility.
Conversely, when optimizing solely for Sharpe ratio, we encountered a different problem: the model learned to exploit the Sharpe calculation by making extreme predictions. Since Sharpe ratio rewards high returns relative to volatility, the model would output unrealistically large values when confident, artificially inflating the mean return. This led to models that showed excellent Sharpe ratios during training but performed poorly on real data, as they had essentially learned to game the metric rather than make meaningful predictions.

4.f.3 Training the model

The model is trained with our custom MLP and the Adam optimizer, which adjusts an independent learning rate for every weight. We iterate for a user-defined number of epochs, feeding the data in mini-batches so that each pass computes a forward prediction, evaluates a composite loss that blends MSE with a Sharpe-based term, and applies Adam's gradient update. The mean batch loss is recorded each epoch as a real-time indicator of convergence.

When an epoch finishes, the network is frozen and scored on a separate validation set. Because these data are unseen during fitting, the resulting Sharpe ratio offers an unbiased preview of how well the model may translate into risk-adjusted trading returns.

To avoid overfitting we add early stopping with a patience of 50 epochs. Any epoch that improves the validation Sharpe is checkpointed; if no further gains appear within the next 50 epochs, training stops automatically and the best checkpoint is re-loaded. In practice most runs plateau long before the maximum epoch count, so early stopping both reduces computation on large financial datasets and captures a model that generalises better by terminating before it memorises training noise.

5 Evaluation methodology and performance metrics

5.a Linear models

5.a.1 OLS

OLS provides the strongest raw signal among the three: the full-sample mean IC is +0.03 and has trended above 0.04 since 2022. Because there is no regularisation, the model fully loads on every input factor, which explains its slightly tighter fit to the long-run drift in returns. That said, the one-step-ahead RMSE remains 17 %, and the predictions still under-react to abrupt swings, so portfolio turnover would be moderate. Figure 2

5.a.2 Ridge CV

Adding an L2 penalty (α optimised via cross-validation) dampens extreme coefficients while keeping all factors in play. The result is a mean IC of +0.03, matching OLS on average but with a visibly smoother and steadily rising profile—to 0.04–0.05 in the last two years. Ridge also shaves a few basis points off peak forecast errors, pulling the rolling RMSE back towards 17 % after the 2020 stress period. The model therefore offers a good balance between stability and responsiveness, making it well-suited for diversified long–short or overlay trades where turnover costs matter. Figure 4

5.a.3 Lasso

With its L1 penalty Lasso retains only 20–30 % of the factors in most folds, giving the sparsest signal but also the weakest. The full-period IC averages +0.02—just above noise—though it, too, has climbed towards 0.04 recently. Forecast dispersion is noticeably lower, so the RMSE stays around 17 % even during volatile windows, but that comes at the cost of muted directional calls: predicted paths smooth over many of the jagged moves in realised returns. On its own, the edge is thin; Lasso is more useful as a robust feature-selector to feed into richer models. Figure 6

5.b XGBoost

As a non-parametric tree-based ensemble, XGBoost was expected to uncover complex, nonlinear interactions within the data. On the held-out test set, it achieved an R^2 of 0.0017, an RMSE of 17.66%, and a Sharpe ratio of 0.0967. While modest in absolute terms, this predictive edge demonstrates that XGBoost is able to extract economically meaningful patterns from firm-level features in a challenging, noisy environment. Its performance is particularly notable given that no custom loss function was used, and that the model was tuned primarily for stability rather than pure Sharpe maximization.

[1] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*,

5.c Multi Layer Perception (MLP)

5.c.1 Evaluation strategy

To evaluate the designed Multi-Layer Perceptron (MLP), we apply it to a held-out test set. This reveals how the model responds to unseen data and delivers both statistical accuracy and practical trading outcomes.

Timed returns.
Portfolio returns are generated by treating the prediction p_t as a position size:

$$\text{timed return} = pi_t * R_{t+1}$$

so a +2 % forecast goes 2 % long, while −3 % opens a 3 % short. The realised profit (or loss) equals this position multiplied by the next-period market move.

Balancing the loss components.
The training loss combines MSE and a Sharpe penalty. Raising the MSE/Sharpe weight ratio tightens the fit to true returns but can dull risk-adjusted performance; lowering it does the opposite, producing bolder forecasts that hunt a higher Sharpe yet may mis-size moves.

Bringing the strategy closer to practice.
Before live trading we would impose conventional safeguards: position caps (e.g. ±100% per asset), long-only variants for restricted mandates, explicit transaction-cost deductions, and portfolio-level risk limits such as sector or VaR constraints.

Performance metrics

MSE: average squared prediction error.
Sharpe ratio: excess return per unit volatility, key for tradability.
Total return: cumulative profit over the test window.
Volatility: standard deviation of timed returns.
Hit rate: percentage of periods with positive return.

Interpretation.
High Sharpe with high MSE indicates the model captures direction but not magnitude. A high hit rate with low Sharpe means many small wins offset by fewer large losses. High return coupled with high volatility signals attractive yet risk-heavy performance.

5.c.2 Parameters and results

Training deep neural networks on full-market equity data is computationally expensive, so we adopted a budget-aware strategy instead of an exhaustive grid search. We restricted tuning to the 100 most actively traded stocks and combined successive-halving with Bayesian optimisation. This pragmatic approach let us explore the space quickly while staying within time and hardware limits—even if it does not guarantee a global optimum.

- Hidden layers : [264, 128, 64] - A progressively narrowing architecture
- Learning rate : 0.001 - Balanced between convergence speed and stability
- Batch size : 128 - Providing sufficient gradient stability
- Dropout rate: 0.55 - Aggressive regularization to prevent overfitting
- Loss weight : MSE (0.8) and Sharpe (0.3) - Prioritizing accuracy while maintaining risk awareness

On the test set this network achieved a Sharpe ratio of 0.095, higher than the market benchmark, confirming that it can generate superior risk-adjusted returns even without a full hyper-parameter sweep.

These settings were tuned for our 100-stock universe; re-tuning would be essential for other sectors (e.g., small-cap value) where the return distribution and liquidity profile differ. **Information Coefficient (IC)** Our model's IC on unseen data is 0.03. In plain terms, the predictions line up with the actual stock returns just a little better than random chance—but that small edge is still valuable. With careful risk and cost control, even a modest IC can translate into extra return for the portfolio. Figure 8

6 Conclusion and Possible Extensions

After rigorous preprocessing and alignment of multiple datasets, we constructed a clean, lagged panel suitable for supervised learning. We evaluated a broad range of models, including linear regression, regularized regressions (Ridge and Lasso), XGBoost, and a neural network (MLP), each chosen to balance interpretability, flexibility, and robustness.

Linear models offered transparent baseline comparisons, and regularization improved their stability in high-dimensional settings. However, nonlinear methods such as XGBoost and the MLP were better suited to capturing the intricate interactions typical of financial data. Among them, the MLP distinguished itself through its ability to jointly optimize for prediction accuracy and risk-adjusted performance, thanks to a custom loss function that blends MSE with a Sharpe-based term. Meanwhile, XGBoost stood out as both a competitive predictor and a powerful feature selector—its ability to identify a compact, high-signal subset of variables proved crucial in improving the performance of downstream models.

Several extensions could be explored in future work. One natural direction is to reformulate the prediction task as a classification problem, using return bins (quintiles) instead of continuous values, which may stabilize learning and simplify interpretation. Another promising extension would be the use of advanced time-series architectures such as the Temporal Fusion Transformer (TFT), which is specifically designed to handle sequential data with known and unknown time-varying covariates. Finally, the deployment of the model in a full portfolio simulation framework—including transaction costs, risk constraints, and long-short position limits—would provide a more realistic assessment of its practical value in an investment context.

7 Appendix

7.a Train-Test Split distribution

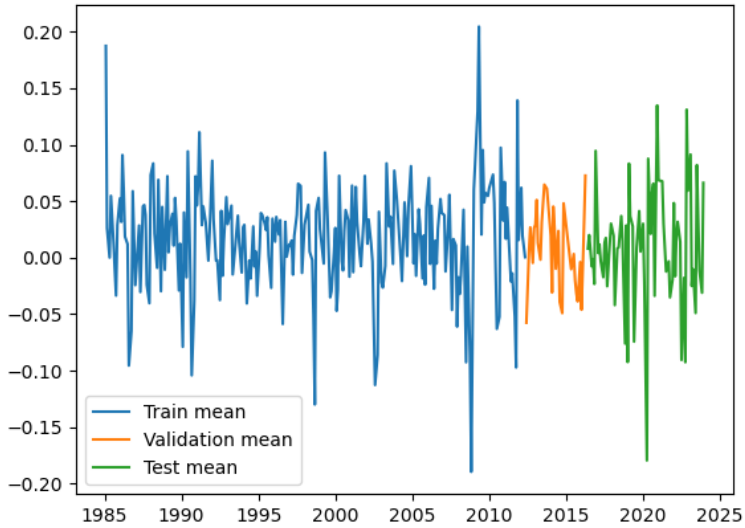


Figure 1: Train-test split distribution

7.b OLS graphs

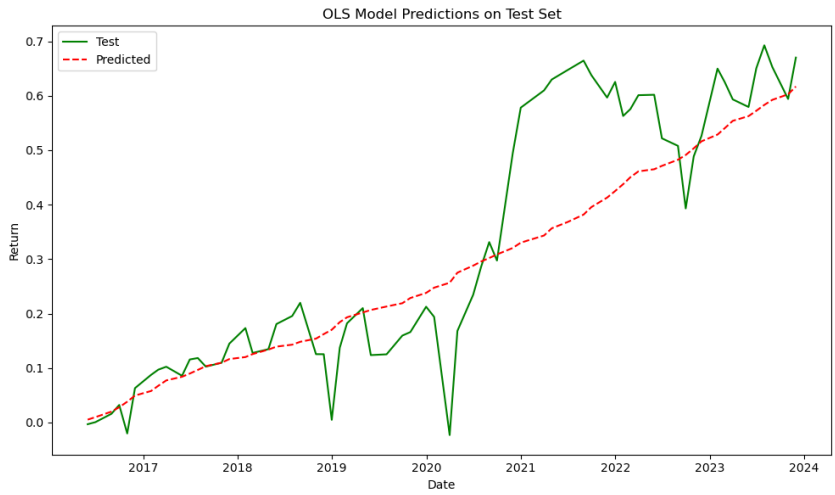
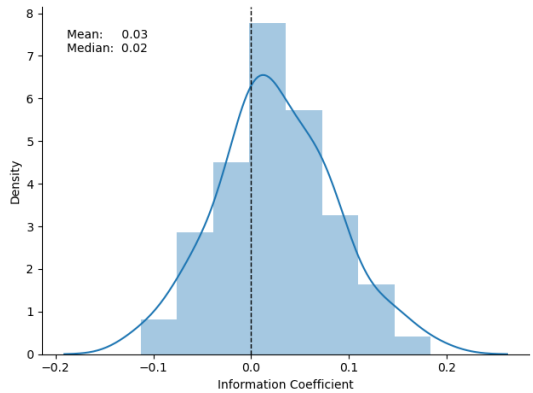
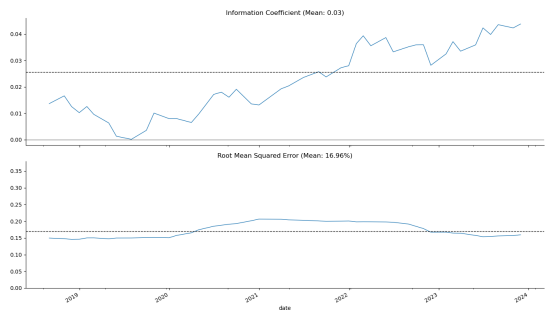


Figure 2: OLS predictions



(a) Distribution of IC



(b) Rolling of IC

7.c Ridge graphs

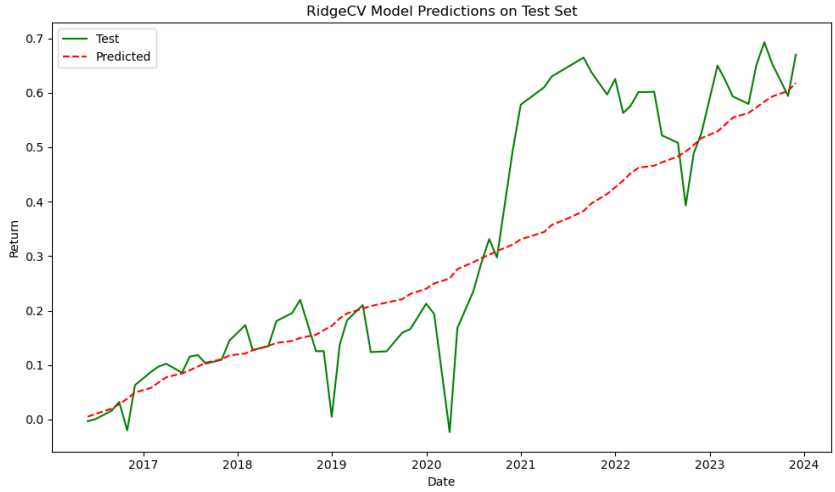
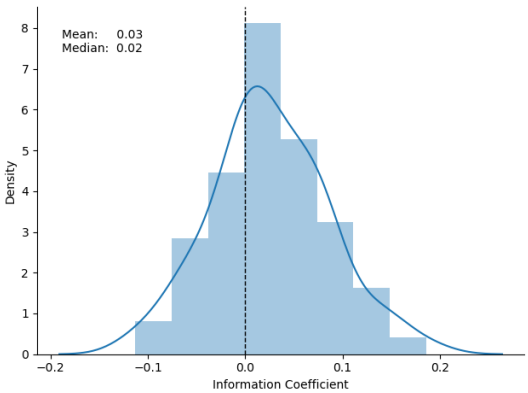
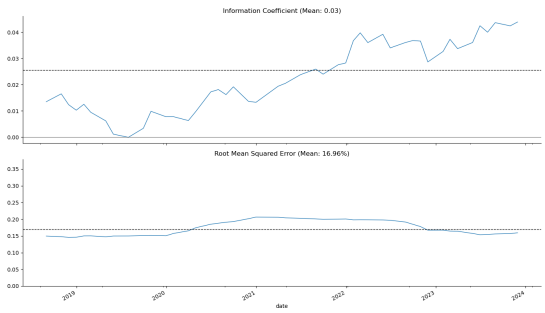


Figure 4: Ridge predictions



(a) Distribution of IC



(b) Rolling of IC

7.d Lasso graphs

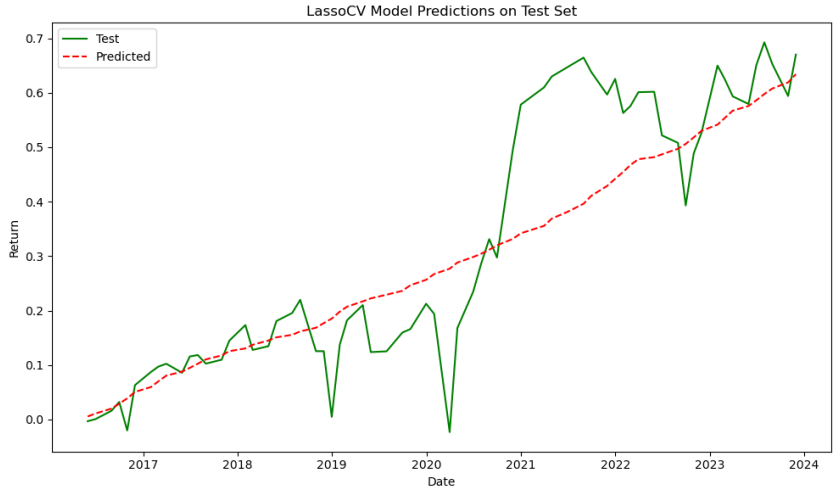
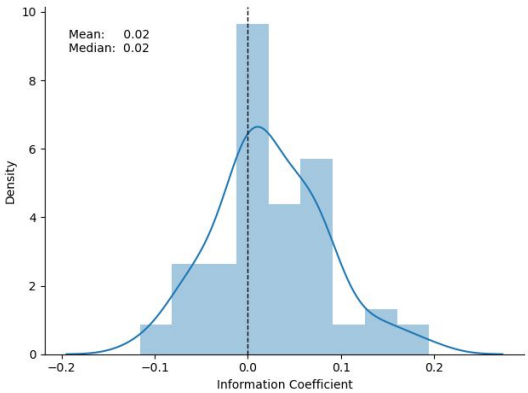
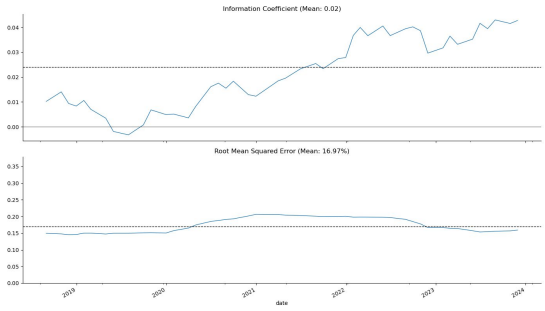


Figure 6: Lasso predictions





(a) Rolling of IC

7.e MLP graphs

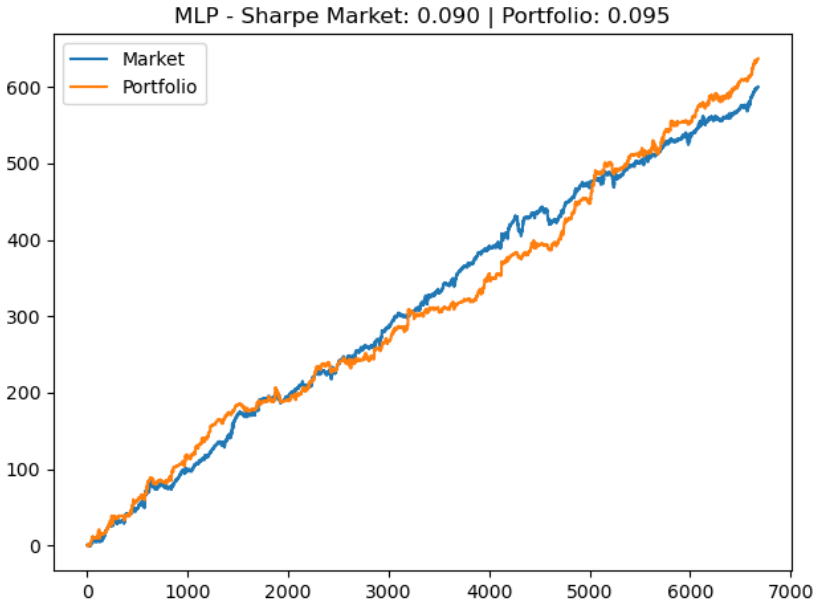
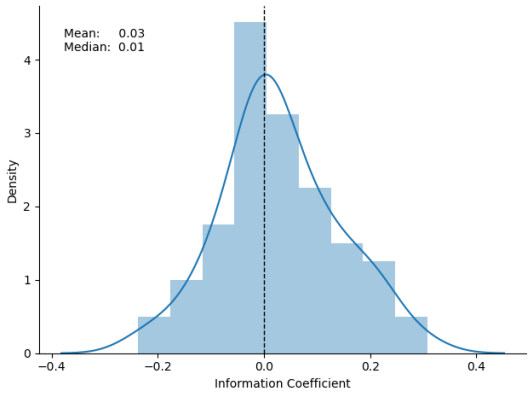
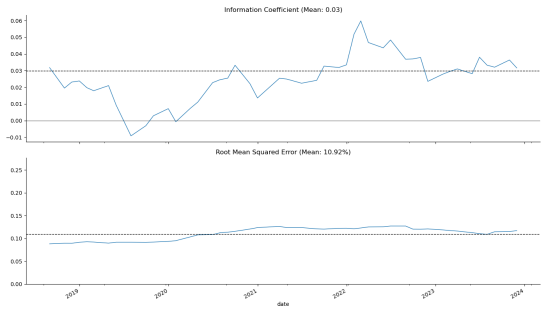


Figure 8: Cumulative Standardized Returns: Market vs. MLP Portfolio



(a) Distribution of IC



(b) Rolling of IC